

DECENTRALIZED UNIVERSAL COMPUTE PROTOCOL

D U C P

Democratizing Sustainable Compute to Power Technological Evolution

Democratized · Sustainable · Abundant
An open marketplace in which verified compute is the work-backed currency.

Technical White Paper

Version 0.1.0 · First Public Release — Request for Comments · June 2026
© 2026 Pawan Singh · All rights reserved · CC BY-NC-ND 4.0

Pawan Singh

Inventor & Author

mr.singhpawan@gmail.com · [linkedin.com/in/singhpawanpreet](https://www.linkedin.com/in/singhpawanpreet)
San Francisco Bay Area, CA

Status of This Document

This is the first public release of the DUCP specification — version 0.1.0, June 2026 — published as a request for comments. DUCP is a design seeking scrutiny and collaborators, not a shipped system. It is developed in the open and invites contribution, while ownership and direction remain with its author through the pre-1.0 phase (see “Stewardship” and “How to contribute,” below, and the Licensing section).

Public versioning

DUCP uses semantic versioning adapted for a specification. The public history begins at 0.1.0:

- **0.MINOR.PATCH** denotes a pre-implementation draft. The MINOR version increments when a substantive revision is accepted; the PATCH version, for editorial corrections.
- **1.0.0** is reserved for a specification frozen against a working reference implementation and a public testnet.

The number signals the maturity of the process, not a claim of completeness. The revisions that preceded this release (2024–2026) were private design iterations by the author, archived for historical interest only; this document supersedes them in full, and nothing in DUCP’s public history predates version 0.1.0.

Stewardship through v1.0

Until the protocol reaches a stable v1.0, DUCP is stewarded by its author, who retains final authority over the specification and its direction. This deliberate, single-steward phase is how the protocol’s identity is shaped coherently before it is set. The reputation-weighted, role-chamber governance described in §12 is the model DUCP is designed to adopt at and after v1.0, when the specification is frozen against a reference implementation; the transition is described in GOVERNANCE.md.

How to contribute

Proposals, critiques, and improvements are welcome through the project repository. CONTRIBUTING.md describes the workflow for issues and changes; GOVERNANCE.md describes how decisions are made. To keep the protocol’s ownership unified during this phase, accepted contributions are made under the DUCP Contributor License Agreement (see Licensing). Substantive changes are proposed, discussed in the open, and versioned as above.

Where help is most needed

The open problems in §14 are the best places to contribute. In particular: trustless energy attestation (the hardest dependency); a formal security analysis (sampling probabilities, slashing calibration, the wash-trading bound, and the game theory of open challenge under collusion); a complete specification of the DVM, the benchmark methodology, and the $\mathbb{U}$ metering model; and cross-paradigm normalization for accelerator-class and, eventually, quantum operations. If you work in any of these areas,

your critique is especially valued.

Abstract

DUCP's purpose is to accelerate technological evolution and the advancement of intelligence by making compute — the resource all of it runs on — a democratized, sustainable abundance: open to anyone to supply or use, plentiful rather than gated, and efficient rather than wasteful. Its central mechanism is to make verified computation itself a real, work-backed currency and a universally redeemable store of value — the unit and money by which an open compute market clears. The currency serves the purpose; it is not the purpose.

The protocol defines a single unit, $\mathbb{U}$ (the Universal Compute Unit; ticker UCU), as a quantity of information processed, its scale fixed by a reference benchmark expressed as information per standard unit of energy. A provider's actual energy never changes their $\mathbb{U}$: the same work earns the same $\mathbb{U}$ on any machine, so the unit is a stable yardstick while efficiency is rewarded outside it. $\mathbb{U}$ is the protocol's native currency — verified work mints money directly, with no separate token — and new $\mathbb{U}$ enters supply only through verified useful work, making the currency elastic with the real compute economy.

Tasks execute once inside a standard environment, the DVM, and are verified by a layered scheme — trusted-execution attestation, zero-knowledge proofs, and sampled re-execution — with instant ledger settlement and retroactive clawback secured by bonded stake. A non-transferable reputation, Standing, unifies honesty, reliability, efficiency, and citizenship, and governs the network through reputation-weighted role chambers bound by a constitution. We present the design in full and state candidly the open problems — chiefly trustless energy attestation — that remain before a 1.0 specification.

Contributions

DUCP's most defensible contributions, situated against prior work in §3, are:

1. **A unit of work that is itself a redeemable currency** — a benchmark-anchored, hardware-neutral measure of classical compute, minted only by verified work, with its scale grounded thermodynamically and its value kept stable by keeping efficiency out of the unit.
2. **A layered, attestation-first verification design** that eliminates routine duplicate computation: run once, check cheaply, re-execute only on challenge.
3. **A work-minted economic model with clean provenance** — one mint source (verified work), transfer-plus-issuance settlement, and burns reserved for fees and fraud correction.
4. **A polity governed by non-transferable reputation**, structurally resistant to capture by capital — the property that most distinguishes DUCP from existing compute networks.

We claim novelty in the **integration** of these into one coherent protocol, and in the specific choice of unbuyable reputation as the governance substrate — not in inventing each primitive from nothing.

Glossary & Notation

Term	Meaning
℥ / UCU	The Universal Compute Unit — the atomic measure of computation and the native currency. Symbol ℥ (logogram); written UCU in plain text; spoken “the U.” One ℥ = a quantity of information processed, scaled by a reference benchmark.
DVM	The DUCP Virtual Machine — the standard, deterministic execution environment every task loads into and runs inside; meters information into ℥.
DUCP Network	The mainnet on which the DVM runs, ℥ is minted and settled, Standing is recorded, and governance takes place.
Standing	A non-transferable, earned reputation (measured in Standing Points, SP) that unifies honesty, reliability, efficiency, and good citizenship; drives reward, routing, lower stake, and governance weight. Cannot be bought or sold.
Roles	Provider (supplies compute), Requester (buys compute), Challenger (open auditor/security), Validator (consensus & settlement checks), Steward (governance), Trader (liquidity).
IR	Intermediate representation — the portable, substrate-independent description of a computation from which ℥ is metered.

DUCP — Protocol Architecture

A single stack: verified compute is metered, verified, settled as currency, and governed by earned reputation

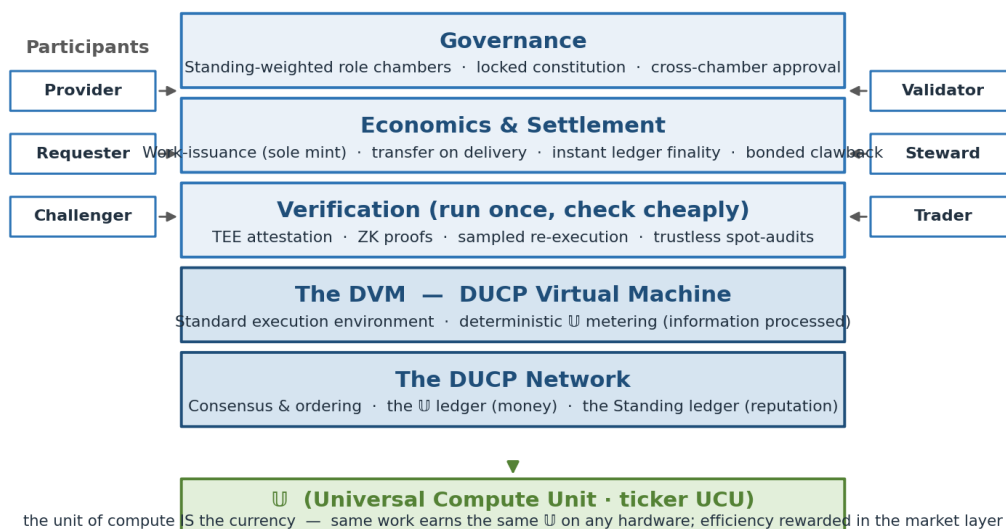


Figure 1 — The DUCP stack: verified compute is metered in the DVM, checked by layered verification, settled as ℥, and governed by non-transferable Standing.

Contents

<i>Status of This Document · Versioning · Contributing</i>	2
<i>Abstract · Contributions · Glossary</i>	3
1. Executive Summary.....	7
2. Purpose & The Problem.....	8
3. Related Work & Positioning.....	10
4. Core Values.....	12
5. Participants & Roles.....	13
6. The Unit — $\mathbb{U}$ (UCU).....	14
7. Currency & Value.....	16
8. Efficiency & the Standing System.....	18
9. Verification.....	21
10. Economics.....	23
11. Task Lifecycle.....	25
12. Governance.....	27
13. Security Model.....	29
14. Honest Limitations & Open Problems.....	31
15. Conclusion & Roadmap.....	31
16. References.....	33
17. Licensing, Ownership & Trademarks.....	34
<i>About the Inventor</i>	34

1. Executive Summary

The Decentralized Universal Compute Protocol (DUCP) is an open, trustless protocol that makes computation a democratized, sustainable abundance and, in doing so, turns compute itself into a real currency of the digital economy. Anyone, on any hardware, anywhere, can supply compute and earn $\mathbb{U}$ (the Universal Compute Unit, ticker UCU) in direct proportion to the useful work they deliver; anyone can spend $\mathbb{U}$ to have computation done; and anyone can hold or trade $\mathbb{U}$ as a store of value whose worth is anchored to the one resource the modern world increasingly depends upon.

The thesis, in four sentences

Compute is the real backing. Every $\mathbb{U}$ in existence was minted by verified, useful computation; nothing is created from nothing, so the currency is backed by work in a way that unbacked fiat and speculative tokens are not.

The unit is the money. $\mathbb{U}$ is at once the protocol's measure of information processed and its native currency. There is no separate token, governance coin, or gas asset.

Efficiency is the keystone value — rewarded, never minted. The $\mathbb{U}$ you earn depends only on the work performed, identical on any hardware. Efficiency is rewarded richly on top — through routing, premiums, and reputation — never inside the unit, where it would invite fraud and hardware discrimination.

Reputation governs, not wealth. Standing — a non-transferable, earned reputation that cannot be bought or sold — steers the protocol, keeping governance a democracy of contribution rather than a plutocracy of holdings.

Compute is the backing the digital economy never had. National currencies are unbacked by anything tangible, and most crypto-assets are backed by nothing but the expectation that someone will pay more later. DUCP starts from the opposite premise: that the soundest possible backing for money is the very thing intelligence, industry, and science all run on. Every $\mathbb{U}$ is minted by verified useful computation and is always redeemable for a defined quantum of compute. The currency is therefore work-backed by construction, and it tends to appreciate as the world's appetite for compute grows.

The unit measures work; efficiency is rewarded around it. $\mathbb{U}$ counts information processed, scaled by a reference benchmark. A provider's own energy use does not change their $\mathbb{U}$: the same task pays the same $\mathbb{U}$ on a state-of-the-art accelerator or a modest CPU. This keeps the unit a stable, comparable yardstick and keeps participation open to all hardware. Efficiency is then rewarded in the layer above the unit: efficient providers win more work through default-on efficiency-preferred routing, command premiums from buyers who pay for clean compute, and accrue reputation that compounds. Efficiency is an additive reward, never a penalty and never a gate.

Work runs once and is verified cheaply. Every task executes inside the DVM — DUCP's standard execution environment — and is checked by a layered scheme: hardware (trusted-execution) attestation as the efficient backbone, zero-knowledge

proofs where a workload permits, and sampled re-execution as a thin fallback. Computation is never routinely duplicated; nodes check a proof or signature at settlement, and full re-execution happens only when a Challenger contests a result. The ledger settles instantly, and fraud discovered afterward is reversed by retroactive clawback from bonded stake.

Reputation, not wealth, governs. A non-transferable, earned standing called Standing unifies the behaviours the protocol cares about — correct results, reliability, efficiency, and good citizenship — into one score that cannot be bought or sold. Standing lowers a participant’s stake, raises their routing priority, and — square-root weighted, capped, and decaying — sets their voice in governance. Because the thing that governs cannot be purchased, DUCP’s governance is structurally resistant, by design, to capture by capital (analyzed in §13).

What is, and is not, solved. DUCP’s contributions are a unit of work that is itself a redeemable currency, a layered verification design that eliminates routine duplication, a work-minted economic model with clean provenance, and a reputation-governed polity that resists plutocracy. The hardest unsolved dependency — trustless attestation of energy, on which the richest efficiency rewards depend — is stated openly in §14, along with the residual risks of TEE compromise, wash-trading, and the two-sided cold start. This is a protocol seeking scrutiny, not a claim of completeness.

2. Purpose & The Problem

2.1 Purpose

DUCP exists to accelerate technological evolution and the advancement of intelligence by making compute — the resource all of it runs on — a democratized, sustainable abundance: open to anyone to supply or use, plentiful rather than gated, and efficient rather than wasteful.

This is the protocol’s north star, stated without mechanism. Intelligence — human and artificial — advances on compute. Scientific discovery, model training and inference, simulation, design, and analysis are all, at bottom, computation, and the rate at which civilization can think is increasingly bounded by the rate at which it can compute. DUCP aims to lift that bound by mobilizing the world’s compute into one open market and by making the efficient use of energy a first-class virtue. Abundance and sustainability are reconciled not by burning ever more energy but by eliminating waste and rewarding efficiency: more compute, performed more cleanly.

Making compute itself a real currency of this economy is the central mechanism by which the purpose is served — but it is a mechanism, not the purpose. A market needs a unit and money to clear; DUCP’s insight is that the soundest possible unit and money are compute itself.

2.2 The Problem

Two problems meet at DUCP’s door, and the protocol is built where they intersect.

Compute is scarce, idle, and centralized — all at once. Demand for computation is outrunning supply: the binding constraint on the advance of artificial intelligence is increasingly access to compute and the power to run it. Yet at the same time, an enormous fraction of the world’s computational capacity sits idle — in personal machines, underused data centers, and edge devices — because no open, trustworthy market exists to put it to work. And what supply is organized is concentrated among a few hyperscale providers, which raises costs, narrows access, and centralizes control over the substrate of intelligence itself. Scarcity, idleness, and centralization are three faces of one problem: the absence of an open, trustless market that can pool heterogeneous compute globally and route work to wherever it can be done cheapest and cleanest.

Money, meanwhile, is backed by nothing real. National currencies float free of any tangible anchor, their value resting on confidence and decree. The dominant crypto-assets are worse: their worth is almost entirely reflexive, sustained by the expectation of future buyers. Even proof-of-work currencies that consume real energy spend it on deliberately useless work, so the cost is real but the product is nothing. The digital economy has no widely used money backed by something both real and universally useful.

The opening. These two problems answer each other. If the world needs an open market for compute, and also needs money backed by something real, then compute itself can be both the good that is traded and the money that trades it. Because computation is verifiable, universally demanded, and increasingly the input to everything, a unit of verified compute is a strong foundation for value: it is produced only by real work, redeemable for the one resource everything needs, and its demand can only grow as intelligence advances. DUCP’s thesis is to seize this opening.

A market for compute is already forming — in dollars

The financialization of compute is underway: regulated compute futures launched in 2026, and live spot markets (e.g. Vast.ai, SF Compute) clear GPU-hours in real time. Today they all price compute in USD per GPU-hour. This validates the thesis that compute is becoming money-like — and sharpens DUCP’s distinct claim. The market is converging on compute as a priced commodity; DUCP proposes the further step of compute as the unit of account itself, hardware-neutral and minted by verified work, so that value is denominated in compute rather than merely measured against the dollar. §3 and §6 develop why a benchmark-anchored unit is the right answer to the objection that “one GPU-hour is not another.”

3. Related Work & Positioning

DUCP draws on, and is distinct from, four streams of prior work. Honesty about lineage strengthens the proposal: the contribution is a specific combination, plus one rare property, not a set of inventions from nothing.

3.1 Where DUCP sits

Stream	Representative prior work	How DUCP differs
Compute / energy as money	Quantum Reserve Token (2025) — one token $\equiv$ one qubit-hour; energy-as-numeraire lineage (Technocracy, 1938).	DUCP meters general classical compute as information processed, hardware-neutral via a benchmark; QRT is quantum qubit-hours, and the energy-money tradition lacks a verifiable, redeemable unit.
Useful-work currencies	Filecoin (storage-minted FIL); Primecoin, Permacoin; Ball-Rosen “Proofs of Useful Work” (2017).	Filecoin mints a separate token for storage; DUCP mints the unit-of-compute itself as the money, uncapped and elastic with the compute economy.
Decentralized verification	TrueBit (interactive bisection $\rightarrow$ single-step re-execution); Ritual (modular ZKML/TEE/optimistic); Gensyn Verde and Prime Intellect TOPLOC (verifying ML).	DUCP composes these into a tiered, attestation-first scheme assigned per workload by the protocol, defaulting to the most efficient sound tier; the components are prior art, the default ordering and economic binding are DUCP’s.
Reputation & governance	Bittensor (stake-weighted Yuma consensus); Akash/Render/io.net (token/stake governance); soulbound tokens (Buterin-Weyl-Ohlhaver, 2022).	Every deployed compute network governs by stake or token. DUCP governs by non-transferable Standing — the soulbound idea applied, for the first time we are aware of, as the governance substrate of a compute economy.

3.2 The novelty claim, stated precisely

DUCP does not claim to be first to propose compute-as-money, useful-work minting, or layered verification; each has identifiable antecedents above. Its claims are two, and they are narrow on purpose:

1. **A novel integration.** No prior system unites an open compute marketplace, a hardware-neutral compute unit that is itself the currency, attestation-first layered verification, and non-transferable reputation governance in one coherent protocol. The contribution is the whole, engineered to be internally consistent — e.g. keeping efficiency out of the unit so the money stays sound while efficiency is still pursued relentlessly.
2. **One genuinely rare property.** Unbuyable, non-transferable reputation as the basis of governance is, as far as we have found, not used by any existing compute

or DePIN network — all of which are stake- or token-weighted. This is the property on which DUCP most clearly departs from the field.

3.3 The hardest objection: is compute a clean unit?

A serious critique of any compute-as-money proposal is that compute is heterogeneous — “one GPU-hour is not another” — so it cannot serve as a clean unit of account. DUCP’s answer is the core of its design (§5–§6): the unit is not a GPU-hour but a quantity of **information processed**, defined over a portable representation and metered deterministically by the DVM, with energy fixing only the ruler’s scale through a consensus benchmark. The same logical work therefore counts as the same $\mathbb{U}$ regardless of the silicon beneath it. Heterogeneous **cost** remains — and is exactly what the market prices, and what efficiency rewards target — but heterogeneous cost is a feature of the market, not a defect of the unit. This is the same separation that lets a kilowatt-hour be a fixed unit while electricity prices vary.

4. Core Values

Ten values constitute the protocol’s constitution of intent. Each is load-bearing, and most map directly onto a mechanism described later. Efficiency is named first because it is the keystone — both an end in itself and the engine by which a finite supply of energy yields an abundance of compute.

Value	What it means in DUCP
Efficiency	The keystone value: doing more useful computation per joule. Efficiency is both the goal and the means of sustainable abundance. It is rewarded everywhere in the incentive layer — routing, premiums, reputation — but never written into the unit, where it would become gameable and discriminatory.
Integrity	Results are correct and honest, every $\mathbb{U}$ represents real work, and records are tamper-proof. Integrity is the value the verification, clawback, and slashing machinery exists to uphold.
Discipline	The rules hold always, with no shortcuts and no discretionary bailouts. $\mathbb{U}$ is minted only for real work; reliability is enforced consistently.
Real work-backed value	Value derives from verified useful work, never from artificial scarcity or unbacked promises. Every $\mathbb{U}$ traces to computation actually delivered.
Open & decentralized	Anyone may participate — supply, consume, verify, trade, or govern — without permission, and no single party controls the network or its money.
Sustainable	Abundance is achieved by eliminating waste and rewarding efficiency rather than by brute consumption of energy. No $\mathbb{U}$ is ever minted for purposeless work.
Trustless	Correctness is established by proof and attestation, not by trusting any participant. The protocol is sound even when most actors are self-interested.
Fair	Identical work earns identical $\mathbb{U}$ everywhere; rules apply uniformly; no hardware class, region, or scale of participant is privileged at the base layer.
Consensus-governed	The protocol’s rules and parameters are set by transparent, on-chain consensus among its participants, not by a controlling foundation.
Democratic	Influence is earned and broadly distributed, weighted to prevent capture by the wealthy or the incumbent. Governance is a democracy of contribution.

Where any two values appear to conflict — most sharply efficiency versus democratization, since the strongest efficiency measurements need capable hardware — the resolution is one discipline observed throughout: **efficiency rules rewards; openness rules access**. They stop fighting once efficiency is an additive bonus and access is universal.

5. Participants & Roles

DUCP is a marketplace, defined by who acts in it. Six roles populate the network. They are roles, not castes: a single entity may hold several at once, and the governance design accounts for that overlap explicitly (§12).

Role	Function
Provider	Supplies compute. Providers accept tasks, execute them inside the DVM on their own hardware, and submit the proof or attestation that authorizes settlement. On verification they receive payment and newly issued $\mathbb{U}$. Any capable hardware may participate without pre-approval.
Requester	Buys compute. Requesters define tasks, escrow payment in $\mathbb{U}$, and receive results. They configure how their own tasks behave on failure (§11) and may pay priority fees for urgency.
Challenger	The open auditor, challenger, and security role. Challengers watch for fraudulent results, vulnerabilities, and misbehaviour; anyone may act as one. They are paid from the fines and clawbacks they trigger and from bounties for responsible disclosure, making security self-funding.
Validator	Runs the network itself: the consensus that orders and finalizes transactions and the cheap proof- and signature-checks performed at settlement. Validators are compensated by a small per-task fee.
Steward	Participates in governance — proposing, deliberating, and voting on parameters and upgrades. Stewardship is exercised through reputation-weighted role chambers rather than by holdings.
Trader	Provides market liquidity for $\mathbb{U}$ as a store and medium of value, buying and selling the currency without necessarily supplying or consuming compute. Traders give holders a voice in governance through their own chamber — representation by participation, not by wealth.

Two further names denote not actors but the system they act within. The **DVM** (DUCP Virtual Machine) is DUCP's standard execution environment — the virtual machine every task loads into and runs inside. It standardizes how computation is expressed and how information processed is counted into $\mathbb{U}$, giving every task a single, portable, substrate-independent definition regardless of the physical hardware beneath it. The **DUCP Network** is the mainnet on which the DVM runs, $\mathbb{U}$ is minted and settled, Standing is recorded, and governance takes place.

6. The Unit — $\mathbb{U}$

6.1 Definition

The Universal Compute Unit, written $\mathbb{U}$ as a logogram and UCU in plain text (spoken “the U”), is the protocol’s atomic measure of computation and, simultaneously, its currency (§7). One $\mathbb{U}$ is a quantity of information processed. Its scale — how much information makes one $\mathbb{U}$ — is fixed by a reference benchmark expressed as information processed per standard unit of energy.

Formal definition

One $\mathbb{U}$ is the amount of information processed that corresponds, at a single consensus reference benchmark, to one standard unit of energy’s worth of work on that benchmark. A provider’s $\mathbb{U}$ for a task equals the information the task processes, normalized to the reference. The provider’s actual energy does not enter the count: the same work yields the same $\mathbb{U}$ on any hardware.

Energy sets the ruler’s scale; information is what is measured. This is the single most important thing to understand about the unit. The reference benchmark is denominated in information-per-standard-energy precisely so the size of one $\mathbb{U}$ is physically grounded — tied, ultimately, to the thermodynamics of computation. But once that scale is fixed, what a provider is paid for is information processed, not energy spent. Two providers who perform the same logical work earn the same $\mathbb{U}$, even if one drew ten times the power. Efficiency does not live in the unit; it lives in the reward layer around it (§8).

A note on the thermodynamic anchor

Grounding the unit in the thermodynamics of computation is a framing, not an operational meter. Landauer’s principle sets a floor ($\approx 2.9 \times 10^{-21}$ J per irreversible bit erased at 300 K) on irreversible operations only — reversible computation can in principle evade it — and practical silicon runs many orders of magnitude above that floor. The benchmark is therefore calibrated to the real efficiency frontier (§6.4), with the thermodynamic limit standing behind it as the ultimate physical reference rather than the day-to-day measure.

The motivation for this “benchmark anchor only” design is decisive. If a provider’s actual energy changed their $\mathbb{U}$, the unit would become hardware-dependent — efficient machines would mint more money for the same work — which is at once a recipe for hardware discrimination, a powerful incentive to commit energy-measurement fraud, and the destruction of $\mathbb{U}$ as a stable yardstick. Keeping energy out of the count dissolves all three problems. Efficiency remains the keystone value; it is simply rewarded outside the unit, where measurement fraud steals nothing and disadvantaged hardware loses nothing.

6.2 $\mathbb{U}$ as Logical Work over a Portable Representation

$\mathbb{U}$ measures logical work, not physical activity on any particular chip. A task is expressed in a portable intermediate representation (IR) — a substrate-independent description of the computation — and $\mathbb{U}$ is a deterministic function of the work that

representation performs. This lets the same task be metered identically on a CPU, a GPU, or a future accelerator: the protocol counts the logical operations the computation entails, not the cycles a given machine spends. Accelerators are credited through an IR aware of their operations (for example, tensor operations counted at their true logical weight) rather than by physically metering the hardware.

The IR is pluggable; the reference is the anchor. DUCP does not hard-code one IR forever. It fixes the requirements a valid IR must meet — determinism, portability, meterability, and verifiability — and lets governance ratify, add, or retire IRs as parameters under constitutional-grade safeguards (supermajority and timelock). This keeps the protocol able to embrace new computational paradigms. A pluggable IR raises a danger — if changing the IR could change what one $\mathbb{U}$ is worth, the IR becomes an instrument of monetary policy — which DUCP closes with a single guardrail.

The reference, not the IR, anchors $\mathbb{U}$

Every ratified IR is normalized to one common value reference. Equivalent work yields equivalent $\mathbb{U}$ regardless of which IR expresses it. The reference benchmark — not any particular IR — is therefore the true anchor of the unit. Adding or retiring an IR is a constitutionally constrained act that must preserve this equivalence, so the meaning and value of $\mathbb{U}$ cannot drift as the IR set evolves.

6.3 The DVM as the Standard Machine

The reference and the IR are abstractions; the DVM is where they become concrete. The DVM is DUCP's standard execution environment — a fully specified virtual machine, defined by specification rather than by any physical device, into which every task loads and within which it runs. It does two jobs at once: it standardizes execution, so a task's behaviour is defined independently of the host hardware, and it meters information processed into $\mathbb{U}$ deterministically, so the unit count is derived by the protocol rather than reported by any participant. Because the DVM is a software standard, it runs everywhere; because it is deterministic, any node can re-derive a task's $\mathbb{U}$ count and confirm it. (The relationship between the DVM as a software standard and the hardware root of trust it can run inside is taken up in §9.)

6.4 A Universally Agreed Benchmark

If the reference benchmark is the true anchor of the unit, the benchmark must be uniform and beyond the reach of any single party. DUCP makes it so through five disciplines:

- **Defined by specification, not hardware.** The benchmark is a canonical reference workload run on the DVM, whose deterministic information-count is paired with a single standard energy figure. Because it is a specification rather than a physical machine, it does not become obsolete as silicon changes.
- **A single consensus on-chain value.** There is exactly one benchmark constant, shared by all nodes by agreement and recorded on-chain. No participant holds a private version of the ruler.

- **Calibrated from reproducible, attested reference runs.** The energy figure is derived from reproducible, attested runs and frontier-efficiency data in the spirit of the Green500 — evidence, not opinion — so the constant reflects the real efficiency frontier.
- **Versioned with synchronized activation.** When the benchmark's data updates, the new value activates network-wide at a synchronized epoch boundary, and every task records the benchmark version under which it was metered. The ruler can be re-calibrated, but never ambiguously.
- **Methodology constitutionally protected.** The method by which the benchmark is defined and calibrated is part of the constitution and cannot be altered by ordinary vote; only the data inputs update, through the locked methodology. This prevents governance from quietly redefining what money means.

The kilowatt-hour analogy. It helps to think of $\mathbb{U}$ the way we think of a kilowatt-hour. A kilowatt-hour is a fixed physical quantum of energy; its definition does not change when the price of electricity moves, and the market prices the cost of producing it separately. $\mathbb{U}$ is analogous: a fixed quantum of computation by definition, with the market pricing the cost of producing it — which varies by hardware, region, and demand — entirely separately. The unit is the measure; the price is the market's business. One frank acknowledgement: because the benchmark is calibrated to the moving efficiency frontier, one $\mathbb{U}$ is frontier-relative rather than a fixed absolute amount of work for all time. This is an accepted tradeoff — it keeps the unit honest about what state-of-the-art computation costs — and the synchronized, methodology-locked versioning above keeps that movement orderly rather than a source of drift.

A worked example

A task. A Requester submits an inference job. The DVM compiles it to a ratified IR and meters its logical work at, say, 4,000 $\mathbb{U}$, recording the benchmark version. The Requester escrows 4,000 $\mathbb{U}$ plus fees.

Execution & settlement. A Provider runs it once inside the DVM and returns a TEE attestation. Nodes check the attestation cheaply. On validity, the 4,000 $\mathbb{U}$ transfers from Requester to Provider, and a small work-issuance — say 1% — mints 40 new $\mathbb{U}$ to the Provider as the work subsidy. The Provider also gains $\approx +4,000$ Standing Points.

The invariant. Whether the Provider ran the job on an efficient accelerator or a modest server, the $\mathbb{U}$ paid is identical: 4,000. Efficiency changes the Provider's cost and their efficiency rewards — not the unit.

7. Currency & Value

7.1 The Unit Is the Currency

In most systems there is a thing of value and, separately, money that measures it. DUCP collapses the two. $\mathbb{U}$ is at once the unit of compute and the currency: when a provider delivers verified useful work, $\mathbb{U}$ is minted directly to them, and to hold or trade $\mathbb{U}$ is to hold or trade money. There is no separate token, no governance coin, and no gas asset. This is not cosmetic — it is what makes the currency genuinely work-

backed, because the act that creates money and the act that creates value are one and the same.

7.2 What Gives $\mathbb{U}$ Its Value

DUCP's value model rests on two pillars: a stable measure and a redeemable anchor.

A stable measure. Because one $\mathbb{U}$ is a fixed quantum of computation by definition (§6), it serves as a stable unit of account — a numeraire. Prices, costs, and value across the network can be expressed in $\mathbb{U}$, and that measuring stick does not stretch or shrink with sentiment, because the definition is anchored to the reference benchmark rather than a market price.

A redeemable anchor. $\mathbb{U}$ is always redeemable for the compute it denominates: one $\mathbb{U}$ can always command its defined quantum of computation, because Providers across the network stand ready to perform work in exchange for it. This redeemability floors the currency's value, enforced by the marketplace through arbitrage rather than any central peg. If $\mathbb{U}$ ever traded below the value of the compute it commands, it would be rational to spend it on compute rather than sell it, and to acquire it to buy compute cheaply — pressure that restores the floor. No manager sets the price; the standing willingness of Providers to honour $\mathbb{U}$ for work does.

Why not an active peg, and why not a free float?

An active peg — a central authority defending a price with reserves — would reintroduce exactly the centralized, fragile control DUCP exists to avoid. A pure free float, with no redeemable anchor, would leave $\mathbb{U}$ as unbacked and speculative as the assets the protocol is meant to improve upon. The stable-measure-plus-redeemable-anchor model is the middle path: market-enforced, manager-free, and genuinely backed.

7.3 $\mathbb{U}$ as Numeraire and Store of Value

$\mathbb{U}$ is designed to be the numeraire of the compute economy — the unit against which other things, including fiat currencies, are measured. Because demand for compute can be expected to rise as intelligence advances, demand for $\mathbb{U}$ should rise with it. An asset that is genuinely backed, strictly limited to creation by real work, and tied to a resource of ever-growing demand has the structural makings of a strong store of value, one that tends to appreciate against unbacked currencies over time.

An honest qualification. $\mathbb{U}$ is stable and appreciating in compute terms — it reliably commands its quantum of computation, and tends to buy more fiat over time. It is not, and does not claim to be, stable against the full basket of non-compute goods and services: the relative price of compute against, say, food or housing will drift with the wider economy, and $\mathbb{U}$ drifts with it. What the protocol guarantees is a sound, work-backed measure and store of compute value, not a universal inflation hedge.

8. Efficiency & the Standing System

8.1 Efficiency Lives in the Reward Layer, Never the Unit

Efficiency is DUCP's keystone value, but the protocol is rigorous about where it belongs. It is defined as $\mathbb{U}$ per joule — useful work delivered per unit of energy spent — and rewarded in the incentive layer above the unit. It is never written into the unit itself, for the reasons in §6: an efficiency term inside $\mathbb{U}$ would be gameable through energy-measurement fraud and discriminatory against older hardware and energy-constrained regions. Keeping efficiency in the reward layer lets the protocol pursue it relentlessly without corrupting the money or excluding anyone.

Three channels reward efficiency, and a fourth principle governs all of them:

- **Efficiency-preferred routing (default-on, overridable).** All else equal, work routes to the more efficient Provider, so efficient hardware structurally wins more volume. Requesters can override for urgency by paying priority fees, and inefficient hardware still participates — especially under high demand — but wins less head-to-head.
- **Market premiums.** Buyers who value clean compute — for sustainability commitments or green-procurement reasons — can pay a premium, and efficient Providers capture it. Demand for verified-efficient compute stacks on top of the base market.
- **Scoring against a single high-efficiency benchmark.** Rather than a sprawling per-device chart, the protocol scores a Provider's attested efficiency against one consensus high-efficiency target that ratchets upward over time, pushing the whole network toward the frontier.
- **Additive, attested, and never punitive.** Every efficiency reward is a bonus on top of the base reward — never a penalty, never an access gate — and is paid only on trustworthy, attested measurement. Where energy can be attested (via a TEE), the bonus is earned; where it cannot, a conservative fallback estimate applies, deliberately low so the absence of measurement yields no windfall. The invariant: no measurement, no bonus, and therefore nothing to fake.

8.2 Standing: Earned, Non-Transferable Reputation

Efficiency is one of several behaviours DUCP wants to encourage; rather than scatter them across unrelated mechanisms, the protocol unifies them in a single reputation system: Standing. Standing is an earned reputation accrued from objective, on-chain events. Its defining property is that it is **non-transferable**: it cannot be bought, sold, lent, or moved between identities. This single property keeps governance a democracy rather than a plutocracy — if reputation could be purchased, the wealthiest would simply buy control. Standing is therefore strictly distinct from $\mathbb{U}$: $\mathbb{U}$ is spendable money; Standing is unspendable reputation. The same good behaviour earns both.

8.3 Standing Points: The Concrete Metric

Standing is measured in Standing Points (SP) — a single number attached to each identity, recorded on a separate, non-spendable ledger. Its dynamics are deliberately legible:

- **Base accrual.** A Provider earns roughly +1 SP per 1 U of verified work — the same scale as the currency, but on a ledger that cannot be spent.
- **Positive adjustments.** Efficiency above the benchmark multiplies accrual upward; sustained reliability (uptime, on-time completion, passing verification) adds to it; and acts of good citizenship — catching fraud, responsibly disclosing vulnerabilities, contributing adopted protocol improvements — earn fixed awards.
- **Penalties.** Fraud, fines, missed deadlines, and downtime subtract Standing, with repeat offences escalating.
- **Decay.** SP decays a few percent each epoch, so reputation reflects current contribution rather than past glory. Decay prevents entrenched incumbency: a participant must keep contributing to keep their standing.

Standing translates into four concrete advantages, forming a virtuous loop in which good behaviour lowers costs, wins work, and grants a voice, which in turn enables more good behaviour:

Standing translates to	Effect
Governance weight	A dampened, capped transform of SP (square-root weighted, with a per-identity cap) sets voting power within a chamber. A hundred-fold contributor wields roughly ten-fold the vote — influence that tracks contribution without letting any one actor dominate.
Lower stake	Higher Standing reduces the stake a participant must lock to take on work, rewarding a proven track record with lower capital cost.
Routing priority	Higher Standing earns more and better work through the matching system, compounding a good reputation into greater volume.
Resilience buffer	Higher Standing grants more tolerance before sanctions, so an established, honest participant is not undone by a single bad day, while a fresh or low-Standing identity has little buffer.

Standing is reputation; U is money

Standing cannot be minted as a reward in the form of new U; doing so would break work-backing by creating money from reputation. Standing's material benefits are funded by redistribution — the fines and clawbacks taken from bad actors flow to good ones — and by market premiums. The reputation system is self-funding and, in steady state, roughly zero-sum; it never inflates the currency.

Because Standing derives only from objective on-chain events — never from subjective ratings — it is trustless and fair, and because a new identity begins with low Standing and high stake requirements, the system is Sybil-resistant: spinning up fresh identities confers no shortcut. Newcomers can climb, rewards are bounded against runaway

accumulation, and a participant whose Standing collapses into bankruptcy is removed, with a deliberately costly path back. All of Standing's constants — the per- $\mathbb{U}$ rate, multipliers, decay, cap, and weighting curve — are governance-tunable, while the non-transferability itself is constitutional and beyond any vote.

9. Verification

9.1 Run Once, Check Cheaply

DUCP's verification is open and permissionless, built around one efficiency principle: computation runs exactly once. A Provider executes a task a single time inside the DVM; every validating node then performs a cheap check — verifying a proof or an attestation signature — as an ordinary part of settlement. This is not re-execution and not duplication; it is the inspection of evidence. Full re-execution happens only when a Challenger raises an open challenge, the exceptional case. In steady state, duplicate work is essentially zero — a direct expression of the no-waste value, and a clean way around the “verifier's dilemma,” since there is no routine re-execution for a lazy verifier to shirk.

Anyone may challenge a result for a bounty, and a challenger posts an anti-spam bond to deter frivolous challenges. The tradeoff DUCP accepts with eyes open is that detecting a compromised execution is reactive — challenger-driven rather than caught by routine duplication — which is precisely why the challenge bounty must remain lucrative and why stake stays bonded long enough for challenges to land (§9.4).

9.2 A Layered Scheme, Strongest Available First

No single verification technique fits all computation, so DUCP layers them and uses the strongest available for each workload:

- **Trusted-execution (TEE) attestation — the backbone.** A hardware root of trust attests that the genuine task ran unmodified and produced the claimed result, with near-zero overhead because the work runs once, natively. The same mechanism does triple duty: it verifies the work, it can attest the energy underpinning efficiency rewards, and it protects the confidentiality of task inputs. This is the most efficient path and, where available, the default.
- **Zero-knowledge proofs — the trustless complement.** Where a workload admits it, the Provider produces a succinct cryptographic proof of correct execution that any node can verify cheaply, with no trust in hardware. ZK proofs are the trustless endgame for the workloads that support them; their cost is the proving overhead. (A ZK proof attests correctness only — it cannot speak to energy.)
- **Sampled re-execution — the thin fallback.** For deterministic workloads on hardware without a TEE, a sample of claimed tasks is re-executed by others. This gives a universal, zero-trust floor at the cost of some duplicated work on the sampled fraction, bounded so the expected penalty for fraud exceeds any gain.
- **Trustless spot-audits — the backstop.** Behind the TEE path sit randomized spot-audits, so even attested results face a standing probability of independent scrutiny. This is the principal defense against a compromised TEE.

Tiers are assigned by the protocol, by workload. A Provider does not choose their own verification tier and a Requester does not specify it. The DVM deterministically assigns each task's tier from its workload type at submission. This is an anti-gaming

property: no Provider can select the weakest check for a sensitive task, and the same task type is verified the same way everywhere. The workload-to-tier mapping is itself a governance-set parameter. A natural consequence is tiered participation — a workload requiring TEE attestation routes only to TEE-capable Providers — and the protocol picks the most efficient sound tier for each workload.

9.3 Software Standard vs. Hardware Root of Trust

It is worth distinguishing two things that are easy to conflate. The DVM is a software standard: a protocol-defined virtual machine that runs everywhere, standardizes execution, and meters information into $\mathbb{U}$. A TEE is a hardware vendor's root of trust: a secure enclave that can attest what ran and, in principle, sign measurements such as energy. The two compose — the DVM runs inside a TEE when one is available — but they answer different questions. Software alone, on an untrusted host, cannot prove to the world that it ran unmodified or measure the host's energy honestly; that requires a hardware root of trust (or, for correctness alone, a zero-knowledge proof, which cannot speak to energy).

9.4 Instant Settlement with Retroactive Clawback

Finality is of the ledger, not of the bond. On a valid proof or attestation, the result is accepted, payment moves, and $\mathbb{U}$ is issued at once: transaction and settlement finality are instant, so the system is fast and usable now rather than probabilistically final over long windows. Soundness is preserved separately, by a bonded stake that remains slashable after settlement. A Provider's stake stays locked for a clawback window; if fraud is later proven, the clawback and an additional fine are taken from that bonded stake, and — critically — an offsetting amount of $\mathbb{U}$ is burned so the fraudulent issuance is removed from supply. The settled transaction is never rewritten; only the fraudster's bond is touched. This is the same separation deployed systems use — a finalized block whose proposer can still be slashed later — not a contradiction in terms.

Downstream holders are never penalized. The offsetting burn is the linchpin of monetary integrity: because the fraudulent $\mathbb{U}$ is removed from supply at the expense of the fraudster's stake, every other holder's money remains fully work-backed. No innocent party absorbs the loss. Challengers and bounties drive detection within the window, which is sized to the realistic detection horizon; a small, bounded tail risk remains after stake unlocks, addressed in §13. The clawback composes cleanly with Standing — the same event that reverses the fraud also strips the offender's reputation.

10. Economics

10.1 Work-Minted, Elastic Supply

New $\mathbb{U}$ is created in exactly one way: as a reward for verified useful work delivered. There is no other mint. The money supply therefore grows in step with the real compute economy — roughly one-to-one with cumulative useful computation — and every $\mathbb{U}$ in existence traces to work actually performed. There is no fixed cap and no halving schedule, because supply is meant to track a growing real economy rather than enforce artificial scarcity. The rate of work-issuance is a governance-tunable dial: set higher early to bootstrap the network, tapering as the economy matures.

10.2 Settlement: Transfer Plus Work-Issuance

When a task settles, two distinct things happen, and keeping them distinct is what gives the currency its clean provenance:

- **The payment is transferred, not burned.** The Requester's escrowed $\mathbb{U}$ moves to the Provider on verified delivery. It is existing money changing hands; nothing is destroyed.
- **New $\mathbb{U}$ enters only through work-issuance.** A small, governance-tunable amount of new $\mathbb{U}$ is issued to the Provider as the reward for the work — the sole source of new money. The Provider thus earns both the Requester's payment (a market price) and the issuance (a work subsidy), reminiscent of a block reward paid alongside a fee.

Clean provenance without burning. Because there is exactly one mint source — verified work — every $\mathbb{U}$ ever created came from real computation, and ordinary transfers merely move that money while preserving its origin. Burns are reserved for genuine supply reduction only: an optional fee-burn for deflationary policy, and the clawback burn that corrects fraud (§9.4).

10.3 Lean Fees and Self-Funding Security

DUCP's fee structure is deliberately lean. A small per-task fee compensates Validators for running consensus and the cheap settlement checks. Security largely funds itself: Challengers are paid from the very fines and clawbacks they cause to be levied, so in steady state the cost of policing fraud is borne by fraudsters rather than honest participants. There is no large standing treasury skimming every transaction. One nuance: ongoing development and rewards for adopted improvements require some funding, handled by a minimal, fee-funded treasury and sponsorship (§12).

10.4 Bootstrap

A compute marketplace faces the classic two-sided cold start. DUCP's bootstrap leans on two assets it uniquely possesses. First, issuance is free: rewarding early Providers with newly issued $\mathbb{U}$ costs no capital. Second, $\mathbb{U}$ has day-one utility: it is redeemable for real compute from the first block, so it is not a speculative token that needs a market price before it is worth anything. The playbook follows: elevated early work-issuance to draw in supply (tapering over time); a single narrow beachhead workload

to concentrate thin early liquidity; and cheap demand seeded by dogfooding — being the network’s own first customer — and by onboarding a few real users with issued- $\mathbb{U}$ credits. There is no premine and no capital-heavy, foundation-paid demand.

The honest crux. Issuance can manufacture supply but it cannot manufacture demand. The bootstrap stands or falls on securing at least one genuine anchor use-case — ideally one the project controls — named plainly among the open problems in §14.

10.5 Urgency: Auction and Surge Pricing

When capacity is scarce, urgent tasks compete through auction and surge pricing: Requesters bid for priority, and the clearing price floats with demand. These bids are transfers of existing $\mathbb{U}$, not new issuance, so urgency pricing redistributes scarce capacity to its highest-value uses and signals Providers to add supply, without inflating the currency. Under normal load, work clears at base prices with efficiency-preferred routing; under congestion, the auction allocates capacity. The tradeoff is variable cost under load, which can price out small Requesters at peak — mitigated by the base market remaining available off-peak and by bidding never being gated to a privileged few.

10.6 Treasury

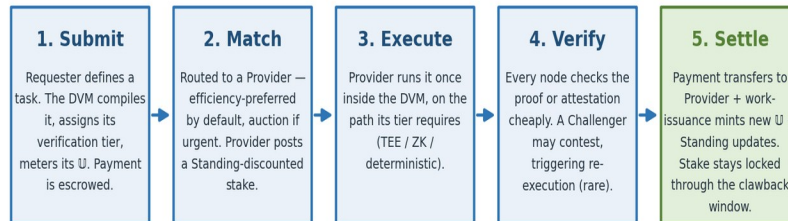
The treasury is minimal by design. It is funded by a small slice of fees and, especially during bootstrap, by sponsorship donations, and allocated retroactively by governance for proven improvements — paying for work after it has demonstrably helped. A vital guardrail makes sponsorship safe: donations buy no governance power and no favouritism. They are pooled and allocated by the same reputation-governed process as any other funds, and sponsors cannot earmark or steer them. This is safe precisely because governance power is unbuyable Standing — a donor cannot convert money into influence even in principle.

11. Task Lifecycle

11.1 The Five Stages

Every task moves through five stages, each with an on-chain artifact, from submission to instant settlement.

The Task Lifecycle — five stages, instant settlement



On a valid proof, settlement is instant and the ledger is final.

Finality is of the ledger, not of the bond: the Provider's stake remains slashable during the clawback window, so fraud found later is reversed without ever rewriting a settled transaction.

Figure 2 — The task lifecycle. Settlement is instant on a valid proof; the Provider's bond remains slashable through the clawback window.

Stage	What happens
Submit	The Requester defines a task. The DVM compiles it, deterministically assigns its verification tier from the workload type, and meters its $\mathbb{U}$ count. The Requester escrows payment in $\mathbb{U}$.
Match	The task is routed to a Provider. Efficiency-preferred routing is the default; an auction governs allocation if the task is urgent. The Provider posts a reputation-discounted stake to claim it.
Execute	The Provider runs the task once inside the DVM, on the path its tier requires — TEE attestation, a ZK proof, or deterministic execution eligible for sampled re-execution.
Verify	Every node checks the proof or attestation cheaply at settlement. Challengers may contest a result on suspicion, triggering re-execution in the exceptional case.
Settle (instant)	On a valid proof, payment transfers to the Provider, work-issuance mints new $\mathbb{U}$ to them, the Validator fee is paid, and Standing updates. The Provider's stake stays bonded through the clawback window.

11.2 Failure Handling

Tasks fail — hardware dies, deadlines slip, results come back invalid — and DUCP separates two questions often conflated: what happens to the Requester's task, and what happens to the Provider who failed it.

The Requester configures their own task's fate. Failed or timed-out tasks default to automatic re-queuing onto another Provider, which keeps the experience seamless and the market liquid. But the Requester may choose otherwise per task: auto-requeue (the default), return-on-failure, or retry-then-return, with optional deadline and cost caps.

The Provider-side penalty is negligence-only and statistical. Accountability is not waivable by a Requester — it is a matter of integrity. A single honest failure (a machine that genuinely died) is not punished; negligence and repeated failure are. The protocol distinguishes the two without adjudicating intent: it relies on the Standing reliability score and the resilience buffer. An occasional failure is absorbed by the buffer; a pattern of failure erodes reliability Standing, which mechanically lowers routing priority and escalates penalties. Judgement is by frequency, not intent — trustless and uniform.

11.3 Task Patterns

Four submission patterns are supported by default, with deadlines forming part of the task specification and enforced through reliability Standing:

- **Single.** One task, one $\mathbb{U}$ count, one escrow — the atomic unit of the marketplace.
- **Batch.** Many independent tasks bundled into one submission to cut overhead, claimable by different Providers.
- **Streaming.** A continuous flow of work drawing from a pre-deposited reserve rather than per-task escrow.
- **Pipeline.** Chained tasks where one task's output feeds the next, each step settling independently — the natural shape for checkpointed workloads such as model training.

12. Governance

12.1 Voting Power: Standing × Role Chambers

Governance power in DUCP derives from earned, non-transferable Standing — never from holdings — which is the structural reason the system resists becoming a plutocracy. Participants are organized into role chambers — Providers, Requesters, Challengers, Validators, and Traders — and a proposal must win cross-chamber approval to pass. This ensures no single interest can capture the protocol: Providers cannot vote themselves a better deal at Requesters' expense, and holders are represented not through wealth but through the Traders chamber, where their stake in the currency gives them a voice.

Within a chamber, voting weight is the square root of a participant's Standing Points, subject to a per-entity cap of a few percent of the total. Square-root weighting damps whales — a hundred-fold contributor wields roughly ten-fold the vote — and the cap bounds any single actor's influence. Standing decay adds an anti-incumbency force, so influence must be continually re-earned. Non-transferability makes the edifice Sybil-resistant: reputation cannot be split across fresh identities to evade the caps. The one residual — an entity spanning several chambers by playing several roles — is handled by aggregating per-entity caps across chambers against a linked identity, on top of the cross-chamber requirement.

Four anti-capture properties, by construction

Anti-plutocracy (power is unbuyable Standing, not wealth); anti-whale (square-root weighting plus caps); anti-incumbent (Standing decay); and Sybil-resistant (non-transferable reputation). These are chosen specifically to preserve democracy, and the governance structure itself is part of the constitution so that it cannot be voted into plutocracy.

12.2 The Constitution: A Locked Core and Tunable Mechanics

DUCP draws a firm line between what may never change and what governance may tune. The identity of the protocol is locked; its mechanics are flexible. The principle is captured in a phrase: **identity locked, mechanics flexible**. Everything that defines what DUCP is — its purpose, values, sound money, and self-rule — is placed beyond the reach of any vote, while the dials that tune how it runs remain in the hands of its participants.

Constitutional (locked, beyond any vote)	Governable (tunable by proposal)
The Purpose	The work-issuance (inflation) rate
The ten Core Values	Benchmark data (via the locked methodology)
The Unit: $\mathbb{U}$ as information, benchmark-anchored, same-work-same- $\mathbb{U}$, efficiency out of the unit	The workload-to-tier mapping
Work-backing: only verified work mints $\mathbb{U}$	Fee levels

Constitutional (locked, beyond any vote)	Governable (tunable by proposal)
Unit-is-currency: no separate token, ever	Standing constants (rates, multipliers, decay, caps, curve)
Standing is non-transferable	Slashing and fine sizes
The democratic governance structure itself	Stake parameters
Redeemability of $\mathbb{U}$ for compute	Surge and auction parameters
The benchmark methodology	Proposal thresholds and improvement-funding allocation

12.3 Proposal Lifecycle

Governance is direct and transparent. Any participant above a small Standing threshold may submit a proposal. A mandatory review window follows, for public deliberation. The proposal then goes to a reputation-weighted, cross-chamber vote and passes only on a cross-chamber supermajority, activating at the next epoch boundary. Deeper or more consequential parameters require a higher Standing threshold to propose and a longer activation delay, so the weight of the safeguard scales with the gravity of the change. There are no privileged proposers and no back rooms: the threshold is participation-earned, the deliberation is public, and the vote is on-chain.

13. Security Model

DUCP's security rests on the composition of mechanisms already introduced — verification, open challenge, stake, clawback, fines, Standing, and the constitution — applied against a concrete threat model. The table summarizes the principal threats and the defenses that meet them; the wash-trading case, which is subtle, is then developed in full.

Threat	Defense
Computation fraud	Layered verification, plus open challenge by Challengers, plus retroactive clawback, plus a punitive fine, plus Standing loss — making fraud net-negative in expectation.
Sybil attack	Non-transferable Standing confers no advantage on fresh identities, and new identities face high stake requirements. Reputation must be earned, not spun up.
Verifier / Challenger collusion	There is no fixed verifier set to bribe; challenge is open and permissionless; per-entity caps and cross-chamber approval limit concentrated influence.
TEE compromise	Randomized trustless spot-audits sit behind the attestation path, and any proven fraud triggers clawback and fines from bonded stake.
Governance capture	Unbuyable reputation-weighted chambers, square-root weighting, per-entity caps, Standing decay, and a locked constitution together make capture by capital structurally very hard.
Spam / denial of service	Per-task fees and stake requirements impose a real cost on flooding the network, and challengers post anti-spam bonds.

13.1 Wash-Trading and Issuance-Farming

The subtlest attack on a work-minted currency is self-dealing: an actor playing both Requester and Provider, submitting work to itself to farm the work-issuance. DUCP meets it with a layered defense and — importantly — bounds the worst case by an invariant rather than relying on detection alone.

- **A structural floor.** Work-issuance is calibrated to sit below the real resource cost of performing the work. A self-dealer therefore burns real energy to earn less than that cost, taking a net loss on each cycle. The economics are adverse to the attacker before any policing begins.
- **Identity separation.** Requester and Provider are Sybil-resisted, distinct identities, and a Requester must hold real, work-earned $\mathbb{U}$ to pay — so the attack cannot be seeded from nothing.
- **Detection backstop.** Anti-collusion heuristics flag linked identities and circular flows, denying issuance and Standing and slashing where self-dealing is detected.

The blast radius is bounded by work-backing

The decisive point is that the work-backing invariant caps the damage. Even a wash-trade that evades every defense still produces real compute: the worst case is a small subsidy leak toward useless-but-real work, never the creation of counterfeit $\mathbb{U}$. The attacker cannot fake the work itself, so the integrity of the money survives even the successful attack. This converts what would be a catastrophic counterfeiting risk in a naive design into, at worst, a minor and bounded subsidy inefficiency.

14. Honest Limitations & Open Problems

A specification's credibility rests as much on the problems it names as on the ones it solves. DUCP is, as of v0.13, a design seeking scrutiny, not a running system, and the following are unresolved. Several gate a 1.0 specification, and the first is the hardest dependency in the architecture.

- **Trustless attested energy does not exist out of the box.** The richest efficiency rewards depend on trustlessly attesting how much energy a task consumed, and no hardware available today provides this: power sensors are unsigned, and confidential-computing modes deliberately suppress fine-grained power telemetry because it is a cryptographic side channel. This is why efficiency rewards are graded and optional — earned where measurement is trustworthy, conservatively estimated where it is not — until the measurement substrate matures. The design is sound without attested energy; it is simply less richly incentivized toward efficiency than it will be once attestation arrives.
- **TEE trust rests on hardware vendors, and TEEs can be broken.** The attestation backbone places trust in silicon vendors' roots of trust, and trusted execution environments have been repeatedly compromised and will be again. The mitigation — randomized trustless spot-audits behind the attestation path, with clawback on proven fraud — reduces but does not eliminate this exposure.
- **Residual wash-trading.** The layered defense and the work-backing invariant bound the damage to a minor subsidy leak rather than counterfeit money, but do not reduce it to zero; sufficiently clever, well-separated self-dealing can still extract a small subsidy at a real-resource loss to itself.
- **The two-sided bootstrap still needs genuine first demand.** Free issuance and day-one redeemability ease the cold start, but cannot manufacture demand. Standing up a liquid two-sided market remains the failure mode of most compute networks, and securing a real anchor use-case is a go-to-market risk the design can mitigate but not remove.
- **Standing-gaming and benchmark governance are attack surfaces.** Any reputation system invites attempts to game it, and the parameters that define the efficiency benchmark are a locus of governance contention. The constitutional protection of the benchmark methodology and the objectivity of Standing inputs are defenses, not guarantees, and both warrant continued adversarial analysis.
- **Cross-paradigm normalization is unsolved.** Normalizing fundamentally different computational paradigms — quantum computation most clearly — to a single classical reference is an open problem. The pluggable-IR design anticipates new paradigms, but how to make a quantum operation commensurable with a classical one in $\mathbb{U}$ terms, without distorting the unit, is not yet answered.

15. Conclusion & Roadmap

DUCP proposes to do for compute what sound money once did for value: anchor it to something real. Its thesis is that a sound currency for an age of intelligence can be

compute itself — verifiable, universally demanded, and produced only by genuine work — and that an open marketplace settling in that currency can mobilize the world’s idle and centralized capacity into a democratized, sustainable abundance. The protocol’s contributions are four: a unit of work, $\mathbb{U}$, that is itself a redeemable currency, with its scale anchored to a reference benchmark and its value kept stable by keeping efficiency out of the unit; a layered, attestation-first verification design that eliminates routine duplicate computation; a work-minted economic model whose every $\mathbb{U}$ has clean provenance; and a polity governed by non-transferable Standing, structurally resistant to capture by capital.

Equally, the protocol is candid about what remains. Trustless energy attestation is a frontier dependency; trusted hardware can be broken; the two-sided market must still be ignited; and cross-paradigm normalization is unsolved. None undermines the core design, but each must be confronted before a 1.0 specification can stand against a working system.

The roadmap follows directly from the open problems, in priority order:

1. Publish a complete, formal specification of the DVM, the reference benchmark methodology, and the $\mathbb{U}$ metering model.
2. Build and open-source a reference implementation — the DVM, the verification layers, the settlement and Standing ledgers, and the governance machinery — and stand up a public testnet.
3. Secure a single genuine beachhead workload and seed two-sided liquidity through dogfooding and issued- $\mathbb{U}$ credits.
4. Mature the efficiency-attestation path — from conservative estimates today toward trustless attested energy as hardware and standards permit — keeping every efficiency reward additive and optional throughout.
5. Formalize the security analysis: sampling probabilities, slashing calibration, the wash-trading bound, and the game theory of open challenge under collusion.
6. Research cross-paradigm normalization, beginning with a ratifiable path for accelerator-class and, eventually, quantum operations.

DUCP is offered in that spirit: a coherent, rigorous proposal for making compute a real currency of the intelligence age — and an open invitation to find its flaws before it is built.

16. References

- [1] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.
- [2] R. Landauer, “Irreversibility and Heat Generation in the Computing Process,” *IBM J. Res. Dev.*, 5(3), 183-191, 1961.
- [3] A. Bérut et al., “Experimental verification of Landauer’s principle,” *Nature* 483, 187-189, 2012.
- [4] E. Ben-Sasson et al., “SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge,” *CRYPTO*, 2013.
- [5] S. Goldwasser, S. Micali, C. Rackoff, “The Knowledge Complexity of Interactive Proof Systems,” *SIAM J. Comput.*, 18(1), 1989.
- [6] J. Teutsch and C. Reitwießner, “TrueBit: A Scalable Verification Solution for Blockchains,” 2017 (arXiv:1908.04756).
- [7] M. Ball, A. Rosen, M. Sabin, P. N. Vasudevan, “Proofs of Useful Work,” *IACR ePrint* 2017/203.
- [8] Protocol Labs, “Filecoin: A Decentralized Storage Network,” 2017; Filecoin Spec, minting model, 2020.
- [9] Y. Sharma, “The Quantum Reserve Token: A Compute-Backed Digital Currency,” arXiv:2503.22056, 2025.
- [10] Bittensor, “Yuma Consensus,” Bittensor documentation, 2024-2025.
- [11] Gensyn, “Verde: Verifiable Machine Learning via Refereed Delegation,” arXiv:2502.19405, 2025.
- [12] Prime Intellect, “TOPLOC: Locality-Sensitive Verifiable Inference,” arXiv:2501.16007, 2025.
- [13] Ritual, “Modular Computational Integrity / Execute-Once-Verify-Many,” technical documentation, 2024-2025.
- [14] E. G. Weyl, P. Ohlhaver, V. Buterin, “Decentralized Society: Finding Web3’s Soul,” SSRN, 2022.
- [15] L. Luu et al., “Demystifying Incentives in the Consensus Computer” (the Verifier’s Dilemma), *ACM CCS*, 2015.
- [16] Intel Corporation, “Intel Trust Domain Extensions (Intel TDX),” *Architecture Specification*, 2023.
- [17] AMD, “AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection,” *White Paper*, 2020.
- [18] NVIDIA, “Confidential Computing on NVIDIA Hopper H100,” *White Paper WP-11459-001*, 2023.
- [19] IETF, “Remote ATtestation procedureS (RATS) Architecture,” *RFC 9334*, 2023.
- [20] M. Lipp et al., “PLATYPUS: Software-based Power Side-Channel Attacks on x86,” *IEEE S&P*, 2021.
- [21] Z. Yang, K. Adamek, W. Armour, “Part-time Power Measurements: nvidia-smi’s Lack of Attention,” *Proc. SC’24*, 2024.
- [22] TOP500 Project, “The Green500 List,” November 2025.
- [23] International Energy Agency, “Energy and AI,” *World Energy Outlook Special Report*, 2025.
- [24] Ethereum Foundation, “EIP-1559: Fee Market Change for ETH 1.0 Chain,” 2019.

17. Licensing, Ownership & Trademarks

DUCP is published openly for reading, citation, and contribution, while ownership and control are retained by the author through the pre-1.0 phase. The terms below are summarized for convenience; the binding terms are the linked license and the Contributor License Agreement.

- **Copyright.** © 2026 Pawan Singh. All rights reserved except as expressly granted below.
- **White paper & specification.** Licensed under Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0). You may read, share, and cite the verbatim, attributed document for non-commercial purposes. You may not publish modified or derivative versions, or use it commercially, without the author's written permission. Full terms: creativecommons.org/licenses/by-nc-nd/4.0/.
- **Implementation rights (pre-1.0).** The author retains all rights in the DUCP protocol and specification. During the 0.x phase, rights to produce or distribute a conforming implementation are reserved. The author intends to release the specification under an open implementation license at v1.0, once it is stable.
- **Contributions.** Contributions are welcomed and accepted only under the DUCP Contributor License Agreement (CLA), which assigns the contributed rights to the author. This keeps the protocol's ownership unified and preserves the author's ability to license or open the protocol in future. Contributors receive a license back to use their own contributions.
- **Trademarks.** "DUCP," "Decentralized Universal Compute Protocol," the U mark, and associated names and logos are trademarks of the author. No trademark rights are granted by the document license; conforming implementations may not use the marks without permission.

This summary is informational and not legal advice. The author intends to have the license, the CLA, and the trademark position reviewed by counsel before publication.

About the Inventor

Pawan Singh is an engineering leader with more than eighteen years in software, currently serving as a Group Engineering Leader at Intuit. He holds an MS in Computer Science from the University of Texas at Arlington and a BS in Computer Science & Engineering, and is a USPTO patent holder with multiple innovation awards. He is based in the San Francisco Bay Area.

Having spent years building platforms that serve millions of users at scale, Pawan came to a conviction about both money and compute. Today's currencies are backed by nothing real, while the world's appetite for computation — the substrate on which all intelligence runs — grows without bound. DUCP is his answer: a protocol in which compute itself becomes a real, work-backed currency and store of value; in which any hardware, anywhere, can participate and earn on equal terms; and in which efficiency is honoured as the keystone of sustainable abundance. Every U represents real work

delivered, and the system is governed not by wealth but by earned reputation — a democracy of contribution for the substrate of the intelligence age.

Contact: mr.singhpawan@gmail.com · [linkedin.com/in/singhpawanpreet](https://www.linkedin.com/in/singhpawanpreet) · San Francisco Bay Area, CA

Suggested citation: Singh, P. (2026). "Decentralized Universal Compute Protocol (DUCP): Technical White Paper." Version 0.1.0, June 2026.